

Sistema de Reservas Salon de Estetica

Esquema de Casos de Uso y Arquitectura con Flujo de Datos

Desarrollo Web — 2026

1. Introduccion

El presente documento describe el esquema de casos de uso y la arquitectura con flujo de datos del Sistema de Reservas para un Salon de Estetica. La aplicacion permite que los clientes consulten servicios disponibles, vean horarios reales y soliciten turnos online. A su vez, el salon puede administrar servicios, horarios, profesionales, bloqueos de agenda y reservas desde un panel interno.

El proyecto se desarrolla como aplicacion web con frontend en React y backend en PHP, usando MySQL como base de datos. El servidor es provisto por el docente, por lo que no se requiere infraestructura local adicional. Git es esencial para el control de versiones.

2. Stack Tecnico

Frontend — Web

- React + TypeScript + Vite
- React Router para navegacion entre paginas
- TanStack Query para consumir la API y manejo de cache
- Zod para validacion de formularios y respuestas
- CSS Modules para estilos

Backend

- PHP (API REST)
- Autenticacion con JWT generado y validado en PHP
- Organizacion por capas: Controller / Service / Repository
- Validacion de datos de entrada en cada endpoint
- Manejo centralizado de errores con respuestas JSON uniformes

Base de Datos

- MySQL
- Migraciones versionadas con scripts SQL
- Relaciones entre entidades: usuarios, servicios, profesionales, reservas, horarios, bloqueos

Infraestructura y Herramientas

- Git — control de versiones esencial para el proyecto
- Servidor provisto por el docente (no se requiere servidor local ni Docker)
- Postman o similar para pruebas de la API durante el desarrollo

3. Roles del Sistema

Rol	Descripcion	Permisos principales
Superadmin	Administrador general del sistema	Gestionar usuarios, admins, servicios, ver todas las reservas, configurar el sistema
Admin	Encargado o dueño del salon	Crear/editar servicios, definir horarios, bloquear agenda, ver y cambiar estado de reservas

User	Cliente final	Registrarse, ver servicios, consultar disponibilidad, reservar y cancelar turnos propios
------	---------------	--

En PHP, los roles se implementan como campo en la tabla usuarios y se validan en cada endpoint verificando el JWT del header Authorization antes de ejecutar la logica de negocio.

4. Casos de Uso

El siguiente cuadro describe cada caso de uso del sistema, indicando el actor que lo ejecuta y el flujo esperado.

Caso de uso	Actor	Descripcion y flujo
Registrar usuario	User	1. El cliente ingresa nombre, email y password. 2. Backend valida formato y que el email no exista en MySQL. 3. Password guardado como hash (password_hash de PHP). 4. Usuario creado con rol USER. Postcondicion: cuenta activa lista para iniciar sesion.
Iniciar sesion (JWT)	User / Admin / Superadmin	1. El usuario ingresa email y password. 2. Backend busca el usuario en MySQL y compara el hash. 3. Si es valido, genera un JWT con el rol embebido. 4. El frontend almacena el token y lo envia en cada request. Postcondicion: sesion activa con acceso segun rol.
Ver servicios disponibles	User	1. El usuario accede al catalogo sin necesidad de login. 2. Backend devuelve solo servicios con active = 1. Postcondicion: lista de servicios visible para el cliente.
Consultar disponibilidad	User	1. El usuario elige un servicio y un profesional. 2. Backend calcula horarios libres segun horario laboral, reservas existentes y bloqueos activos en MySQL. Postcondicion: lista de turnos disponibles.
Reservar turno	User	1. El usuario selecciona horario, servicio y profesional. 2. Backend valida: no pasado, dentro de horario laboral, servicio activo, sin conflicto, sin bloqueo. 3. Si pasa todas las validaciones, crea la reserva con estado pending. Postcondicion: reserva creada y horario ocupado.
Ver mis reservas	User	1. El usuario autenticado consulta su historial. 2. Backend filtra solo las reservas del usuario del JWT. Postcondicion: lista de reservas propias.
Cancelar reserva propia	User	1. El usuario selecciona una reserva activa propia. 2. Backend verifica que sea del usuario del JWT y este activa. 3. Cambia el estado a cancelled y libera el horario. Postcondicion: horario disponible nuevamente.
Gestionar servicios	Admin / Superadmin	1. El admin crea, edita o desactiva un servicio. 2. Backend valida el rol del JWT antes de proceder. 3. Persiste el cambio en MySQL. Postcondicion: catalogo actualizado.
Definir horarios laborales	Admin / Superadmin	1. El admin define dias y rangos horarios por profesional. 2. Backend guarda el horario laboral en MySQL. Postcondicion: disponibilidad calculable para ese profesional.
Bloquear horarios	Admin / Superadmin	1. El admin crea un bloqueo con rango de fecha/hora y motivo. 2. Backend impide reservas en ese rango para ese profesional. Postcondicion: horario bloqueado no disponible para clientes.

Ver reservas del salon	Admin / Superadmin	1. El admin consulta todas las reservas del salon. 2. Backend no filtra por usuario (a diferencia del cliente). Postcondicion: vision global de la agenda.
Cambiar estado de reserva	Admin / Superadmin	1. El admin cambia el estado de una reserva (pending -> confirmed -> completed / cancelled). 2. Backend valida la transicion de estado permitida. Postcondicion: estado actualizado en MySQL.
Gestionar usuarios / admins	Superadmin	1. El superadmin crea, edita o desactiva usuarios y admins. 2. Puede asignar o cambiar roles. Postcondicion: base de usuarios actualizada.
Configuracion general	Superadmin	1. El superadmin modifica configuraciones del sistema. Postcondicion: configuracion aplicada globalmente.

5. Arquitectura y Flujo de Datos

El backend en PHP se organiza por capas siguiendo el patron MVC. A continuacion se describe cada capa y el flujo completo de una request.

5.1 Capas del Backend

Capa	Tecnologia	Responsabilidad
Router	PHP puro o micro-framework	Mapea las rutas HTTP a los controllers correspondientes.
Middleware	JWT Middleware PHP	Verifica el token JWT en el header Authorization, extrae el rol y bloquea si no tiene permisos.
Controller	PHP — clases Controller	Recibe la request, valida datos de entrada y llama al service correspondiente.
Service	PHP — clases Service	Contiene las reglas de negocio. Es la capa mas importante y la mas testeada.
Repository	PHP — clases Repository	Accede a MySQL mediante PDO o similar. Encapsula todas las consultas SQL.
Model	PHP — clases Model	Representan las entidades del sistema (Usuario, Reserva, Servicio, etc.).

5.2 Flujo de datos — Reservar turno

Ejemplo del flujo completo de la request mas compleja del sistema:

- 1. Frontend (React):** El usuario selecciona servicio, profesional y horario. TanStack Query envia POST /api/reservations con el JWT en el header Authorization.
- 2. Router PHP:** El router mapea la URL al ReservationController.
- 3. JWT Middleware:** Se valida el token JWT, se extrae el user_id y el rol. Si es invalido, responde 401. Si no tiene permisos, responde 403.
- 4. ReservationController:** Recibe el JSON, valida que los campos requeridos esten presentes y llama a ReservationService->create().
- 5. ReservationService:** Aplica todas las reglas de negocio: no puede ser en el pasado, debe estar dentro del horario laboral, el servicio debe estar activo, no puede haber conflicto con otra reserva, no puede haber bloqueo activo.
- 6. AvailabilityService:** Consultado por ReservationService para verificar disponibilidad del slot contra MySQL.
- 7. ReservationRepository:** Si pasa todas las validaciones, ejecuta el INSERT en MySQL con estado pending.
- 8. Respuesta:** El Controller devuelve JSON con 201 Created y los datos de la reserva.
- 9. Frontend:** TanStack Query invalida la cache de disponibilidad y muestra confirmacion al usuario.

5.3 Estructura de carpetas del Backend

```
backend/  
index.php – punto de entrada, carga el router  
router.php – define todas las rutas de la API  
middleware/ – JwtMiddleware.php, AuthMiddleware.php  
controllers/ – ReservationController.php, AuthController.php...  
services/ – ReservationService.php, AvailabilityService.php...  
repositories/ – ReservationRepository.php, UserRepository.php...
```

models/ – User.php, Reservation.php, Service.php...
config/ – database.php (conexion PDO a MySQL)
migrations/ – scripts SQL versionados para crear las tablas
.htaccess – redirige todas las requests a index.php (Apache)

6. Entidades Principales (MySQL)

Entidad / Tabla	Campos principales	Relaciones
usuarios	id, nombre, email, password_hash, rol (user/admin/superadmin), creado_en	1 usuario -> N reservas
servicios	id, nombre, descripcion, duracion_minutos, precio, activo	1 servicio -> N reservas
profesionales	id, nombre, activo	1 profesional -> N horarios 1 profesional -> N reservas 1 profesional -> N bloqueos
horarios_laborales	id, profesional_id, dia_semana, hora_inicio, hora_fin	N:1 con profesionales
reservas	id, usuario_id, servicio_id, profesional_id, fecha_hora_inicio, fecha_hora_fin, estado, creado_en, cancelado_en	N:1 con usuarios N:1 con servicios N:1 con profesionales Estados: pending, confirmed, cancelled, completed
bloqueos	id, profesional_id, fecha_hora_inicio, fecha_hora_fin, motivo	N:1 con profesionales

7. Reglas de Negocio

Estas reglas se validan en la capa Service del backend PHP:

- No se pueden reservar turnos en el pasado.
- No se puede reservar fuera del horario laboral del profesional.
- No se puede reservar un servicio inactivo.
- La duracion del turno depende del servicio elegido.
- No se puede reservar si el profesional ya tiene otra reserva en ese rango horario.
- No se puede reservar si el horario esta bloqueado.
- Una reserva activa puede cancelarse.
- Una reserva cancelada no puede volver a cancelarse.
- Cancelar una reserva libera el horario.
- Un usuario solo puede ver y cancelar sus propias reservas.
- Un admin puede ver todas las reservas del salon.
- Solo admin o superadmin pueden crear, editar o desactivar servicios.
- Solo admin o superadmin pueden definir horarios laborales.
- Solo admin o superadmin pueden bloquear horarios.

8. Casos de Prueba

#	Caso	Resultado esperado
1	Reservar turno disponible dentro del horario laboral	Reserva creada correctamente. Horario deja de estar disponible.
2	Reservar turno ya ocupado por otro cliente	Sistema rechaza la reserva con error de conflicto.
3	Reservar fuera del horario laboral del profesional	Sistema rechaza la reserva.
4	Reservar en fecha/hora anterior al momento actual	Sistema rechaza la reserva.
5	Cancelar una reserva activa propia	Estado cambia a cancelled. Horario vuelve a estar disponible.
6	Cancelar una reserva ya cancelada	Sistema rechaza la operacion.
7	Cancelar reserva de otro usuario	Sistema rechaza por permisos (403 Forbidden).
8	User intenta crear/editar un servicio	Sistema rechaza por permisos (403 Forbidden).
9	Login con credenciales incorrectas	Sistema responde 401 Unauthorized.
10	Request sin JWT a endpoint protegido	Sistema responde 401 Unauthorized.

Control de Versiones

Git es una herramienta esencial en este proyecto. Se utilizara para:

- Mantener el historial de cambios del codigo.
- Trabajo colaborativo entre los integrantes del equipo.
- Ramas (branches) para desarrollar funcionalidades de forma aislada.
- Pull requests para revisar codigo antes de integrar cambios.
- El repositorio se aloja en GitHub.